

Sistema Automatizado de Gestión y Control de Gastos Comunes (DAS) Documento Arquitectura de Software Versión 1.0

Integrantes: Nahuel Bustos
Bastian Cisternas
Javiera Olivares

Profesor: Ricardo Pino

Asignatura: Ingeniería de Software

07 de abril del 2026

Identificación de Documento

Identificación	001
Proyecto	Sistema automatizado de gestión y control de gastos comunes.
Versión	1.0

Documento mantenido por	Nahuel Bustos - Bastian Cisternas - Javiera Olivares
Fecha de última revisión	—
Fecha de próxima revisión	07 - 04 - 2026

Documento aprobado por	Ricardo Pino
Fecha de última aprobación	—

Historia de Revisiones

Fecha	Versión	Descripción	Autor
02 - 04 - 2026	1.0.0	Introducción	Todos los integrantes
03 - 04 - 2026	1.0.1	Visión del Sistema	Todos los integrantes
14 - 05 - 2026	1.0.2	Estilos arquitectónicos, Vistas 4+1	Todos los integrantes
17 - 05 - 2026	1.0.3	Metodología y gestión	Bastian Cisternas

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

El edificio "El mirador", ubicado en zona céntrica, con 160 departamentos y 20 pisos de altura está presentando una gestión financiera crítica de gastos comunes debido a morosidad de los residentes en lapsos de pago, uso de procesos manuales para la organización administrativa lo que consumía mucho tiempo y riesgos.

Actualmente, el 60% de los residentes son propietarios y el 40% arrendatarios, lo que complica la logística administrativa. La ineficiencia generada gracias a los cobros morosos ha afectado el flujo de caja, retrasando mantenciones fundamentales de seguridad. La falta de un sistema automatizado para identificar morosos genera desconfianza entre los residentes y administración.

1.2. Propósito

El propósito de este proyecto es desarrollar e implementar una solución tecnológica que automatice la gestión financiera en el recinto. Se busca garantizar la precisión en la cobranza, asegurar una transparencia en el uso de fondos y permitir en tiempo real el acceso a información financiera tanto como a los administradores como para los residentes, así optimizando la estabilidad financiera del edificio.

1.3. Ámbito

Nuestro sistema abarca la gestión total de los 160 apartamentos, incluyendo el registro de propietarios, arrendatarios y personal. Se incluirán módulos para el cálculo automático de cuotas, registro de pagos, emisión de comprobantes o boleta y un módulo específico para la gestión de morosidad con alertas automáticas. Además se integrará un canal de comunicación destinado a la comunidad para solicitudes y quejas.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
HTTPS	Es un protocolo que cifra la comunicación entre un navegador web y un sitio web, proporcionando una conexión segura. Garantiza que los datos transmitidos entre el usuario y el sitio web no puedan ser interceptados ni manipulados por partes no autorizadas.
RBD	Base de Datos relacional organiza la información en tablas(filas, columnas) que se vinculan entre sí mediante claves primarias y foráneas gestionando datos estructurados con alta integridad
API	Interfaz de Programación de Aplicaciones es un conjunto de reglas y protocolos que permite dos aplicaciones de software se comuniquen entre sí intercambiando datos y funciones

MVP	Producto mínimo viable es la versión más simplificada y funcional de un producto que incluye solo lo esencial para resolver el problema central del usuario
------------	---

1.5. Resumen ejecutivo

Este proyecto nace como una respuesta a la crisis administrativa del edificio “El mirador”, donde la morosidad financiera, falta de transparencia y un sistema anticuado amenaza la seguridad de los residentes. Se propone el desarrollo de un sistema web de gestión que automatiza el ciclo del cobro y así se mejore la comunicación entre la comunidad.

Con su implementación, se espera reducir las tareas manuales, agilizar la recepción de pagos mediante pasarelas seguras como Webpay y recuperar la confianza entre los usuarios y administrativos, mediante haya una visualización de datos en tiempo real, en busca de una mejor calidad de vida y convivencia.

1.6. Arquitectura del sistema

-
- Interfaz intuitiva, compatible con dispositivos móviles y computadoras.
- Lógica de negocio para el cálculo de cuotas y gestión de roles,
- Base de datos relacional para asegurar integridad en los registros de pago y datos de los residentes y trabajadores.
- Implementación de protocolos HTTPS para la protección de transferencias financieras.

2. VISIÓN DEL SISTEMA

2.1 Descripción general del sistema

Software que administra los pagos de gastos comunes del edificio “El Mirador” teniendo una base de datos con los pagos ingresados registrando con fecha , numero run y nombre del residente el cual efectúa el pago y activando una alerta de morosos que se registra en el sistema si no se efectúa el pago después de la fecha acordada de este modo teniendo un vasto registro el cual resolverá el problema de liquidez a la administración del edificio “El Mirador”

2.2 Objetivos del sistema

El objetivo del sistema es tener un mayor control ante los propietarios que pagan y los morosos teniendo un registro claro del día de pago y en caso de no pago los días de atraso lanzando alertas a los residentes los cuales aún no pagan de esta manera tener un registro claro de la información la cual será una gran solución al problema para la administración del edificio económicamente evitando pérdidas y teniendo un flujo de dinero rentable

2.3 Requerimientos funcionales

- **RF-01**

- Creación de residentes:**

- El sistema debe permitir el crear los datos de los propietarios, arrendatarios y trabajadores (160 total, 64 arrendatarios, 96 propietarios y personal 5 personas (nombre, contacto, departamento, tipo de residencia (propietario o arrendatario) y datos de contacto en caso de emergencia, entre otros.

- Ámbito funcional:** Gestión de residentes.

- Actores y roles relacionados:** Administrador.

- **RF-02**

- Eliminación de residentes:**

- El sistema debe permitir eliminar los datos de los propietarios, arrendatarios y trabajadores.

- Actores y roles relacionados:** Administrador.

- **RF-03**

- Modificación de residentes:**

- El sistema debe permitir modificar los datos de los propietarios, arrendatarios y trabajadores (160 total, 64 arrendatarios, 96 propietarios y personal 5 personas (nombre, contacto, departamento, tipo de residencia (propietario o arrendatario) y datos de contacto en caso de emergencia, entre otros.

- Ámbito funcional:** Gestión de residentes.

- Actores y roles relacionados:** Administrador.

- **RF-04**

- Cálculo de gastos comunes:**

- El sistema debe calcular automáticamente el cobro mensual dividiendo proporcionalmente los gastos de servicios (agua, luz, gas, sueldos) entre las unidades.

- Ámbito funcional:** Gestión de pagos.

- Actores y roles relacionados:** Administrador.

- **RF-05**

- Pago Online (Webpay):**

- El sistema debe tener la integración de webpay para poder permitirle pagar a los propietarios (gastos comunes) y arrendatarios (arriendo y gastos comunes) directamente de la app web.

- Ámbito funcional:** Gestión de pagos.

- Actores y roles relacionados:** Propietario, Arrendatario.

- **RF-06**

Control de Morosos:

El sistema debe identificar departamentos con deudas y aplicar intereses o multas dependiendo de nuestro cliente.

Ámbito funcional: Gestión de pagos.

Actores y roles relacionados: Administrador.

- **RF-07**

Emisión de recibos:

Debe generar automáticamente comprobantes de pago en PDF tras ser confirmada la transacción.

Ámbito funcional: Gestión de Pagos.

Actores y roles relacionados: Administrador, Residentes.

- **RF-08**

Envío de reporte:

El sistema debe tener un módulo de solicitudes y quejas, el que permitirá a los usuarios reportar eventualidades mediante una descripción detallada.

Ámbito funcional: Solicitudes y quejas

Actores y roles relacionados: Residentes

- **RF-09**

Organización de tareas:

El sistema debe tener un dashboard para la administración y supervisión de reportes, donde el administrador los supervisa, establece orden de prioridad y designa personal técnico para obtener solución al problema.

Ámbito funcional: Solicitudes y quejas.

Actores y roles relacionados: Administrador.

- **RF-10**

Aviso de solución:

El sistema debe tener un canal de comunicación diseñado para informar el cierre de incidencias, brindando al copropietario visibilidad del cumplimiento de sus solicitudes.

Ámbito funcional: Solicitudes y quejas

Actores y roles clave: Administrador y residentes.

- **RF-11**

Notificaciones automáticas:

El sistema debe enviar correos electrónicos avisando a los residentes el plazo de pagos para que no haya atrasos.

Ámbito funcional: Gestión de pagos.

Actores y roles relacionados: Administrador, Residentes.

- **RF-12**

Generación de recibos:

El sistema debe dar reportes detallados sobre la ejecución financiera y estados de cuenta en tiempo real, permitiendo visualizar hacia dónde van los fondos y el saldo disponible en caja.

Ámbito funcional: Consultas e informes.

Actores y roles relacionados: Administrador.

- **RF-13**

Generación de Consultas e Informes:

El sistema debe permitir la generación de informes sobre aspectos de la gestión del edificio tales como gestión de residentes, personal, instalaciones etc todo de cara a la administración.

Ámbito funcional: Consultas e informes

Actores y roles relacionados: Administracion

2.4 Requerimientos no funcionales

- **RNF-01**

Seguridad de datos:

Los datos financieros deben estar encriptados para proteger las transacciones y datos de los usuarios.

Ámbito: Seguridad

Actores y roles relacionados: Todos los usuarios.

- **RNF-02**

Atención continua

El sistema debe estar siempre disponible 24/7 y navegación estable garantizada.

Ámbito: Disponibilidad

Actores y roles relacionados: Todos los usuarios

- **RNF-03**

Intuitividad:

El sistema debe tener una interfaz intuitiva, diseñada bajo principios de experiencia de usuarios, garantizando que cualquier usuario pueda navegar en el sistema sin necesidad de conocimientos técnicos previos.

Ámbito: Usabilidad

Actores y roles relacionados: Residentes, administrador.

- **RNF-04**

Flexibilidad:

Sistema escalable y adaptable a cualquier entorno nuevo o complejo residencial.

Ámbito: Escalabilidad.

Actores y roles relacionados: Administrador.

- **RNF-05**

Tiempo de respuesta:

Optimización de alto rendimiento y baja latencia, garantizando tiempos de respuesta inmediatos en consultas de estados de cuenta, incluso en momentos de alta demanda.

Ámbito: Velocidad y agilidad.

Actores y roles relacionados: Todos los usuarios.

- **RNF-06**

Multiplataforma:

La aplicación debe ser accesible desde distintos navegadores web y dispositivos móviles.

Ámbito: Compatibilidad.

Actores y roles relaciones: Todos los usuarios.

- **RNF-07**

Trazabilidad:

La plataforma ofrece vitrina financiera donde se puedan ver los movimientos del dinero al instante, para eliminar dudas y construir confianza en la comunidad.

Ámbito: Transparencia.

Actores y roles relacionados: Administrador, propietarios.

- **RNF-08**

Integridad de datos:

El sistema cumple reglas inteligentes que bloquean cobros duplicados y garantizan saldos 100% reales y actualizados.

Ámbito: Fiabilidad.

Actores y roles relacionados: Administrador.

- **RNF-09**

Concurrencia:

Sistema robusto que garantiza fluidez total incluso cuando toda la comunidad se conecta simultáneamente.

Ámbito: Capacidad.

Actores y roles relacionados: Todos los usuarios.

- **RNF-10**

Cumplimiento normativo

El sistema se debe actualizar automáticamente ante cualquier cambio en la legislación chilena, asegurando que la administración del edificio cumpla siempre las obligaciones legales sin errores.

Ámbito: Legalidad.

Actores y roles relacionados: Administrador.

2.5 Supuestos y dependencias

Supuestos:

-Hardware: Se da por supuesto que el sistema hardware donde se instalará el software cuenta con los requisitos mínimos para ejecutarlo correctamente sin errores ni fallos de rendimiento

Dependencias:

-Webpay Se depende de Webpay la cual funciona mediante una API REST para efectuar los pagos de los propietarios y arrendatarios

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Microservicios

Este estilo arquitectónico organiza la aplicación como un conjunto de servicios pequeños, independientes y desplegables de forma autónoma, cada uno ejecutando un proceso de negocio específico y comunicándose por APIs bien definidas

3.2. Justificación del estilo según el contexto del sistema

Implementación de módulos externos al sistema
(sistema de pago y notificaciones)

3.3. Patrones de diseño aplicados

Uso de patrón MVC (Modelo - Vista - Controlador), código limpio, separación de datos, lo que nuestro usuario ve y lógica de conexión.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

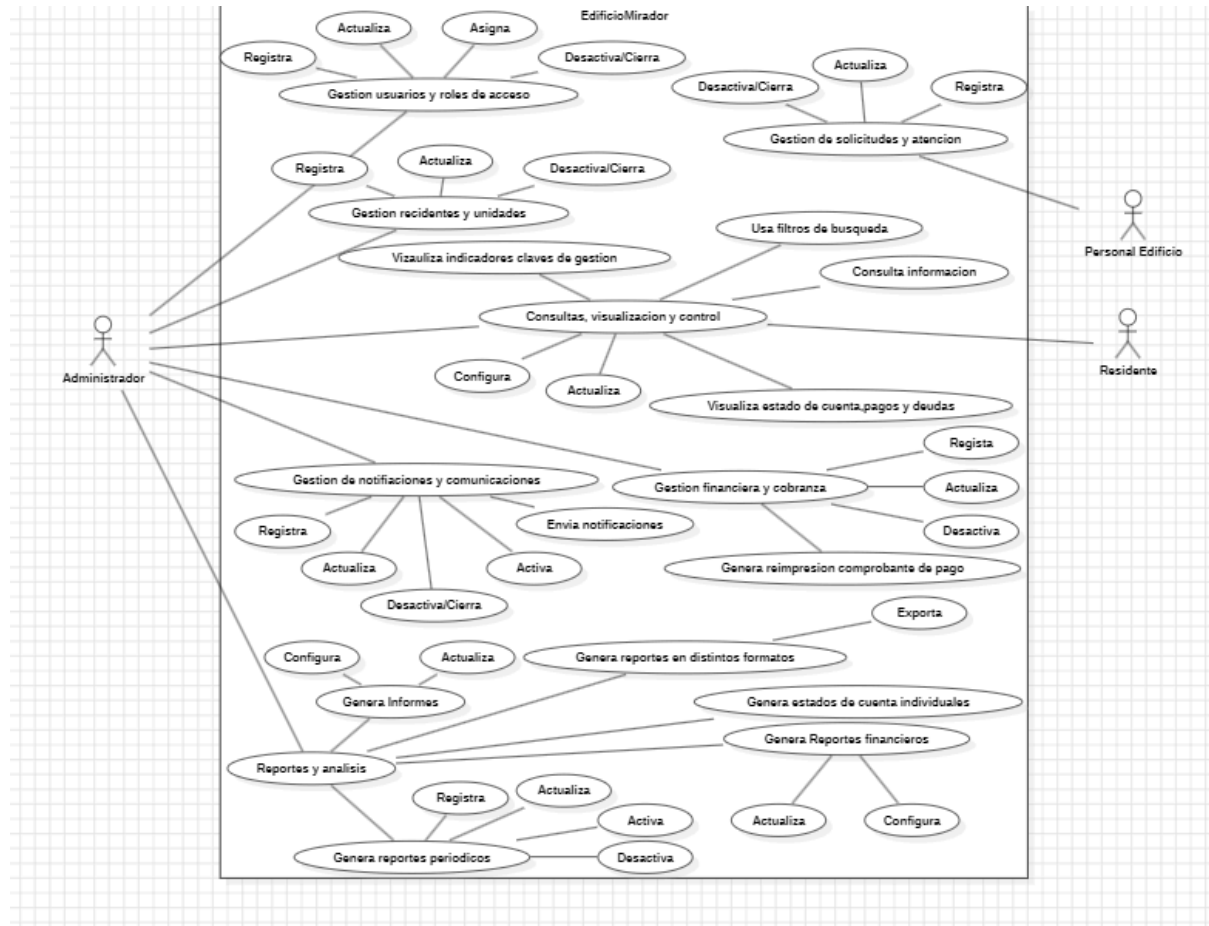
4.1.1. Propósito:

Mostrar cómo interactúan los actores con el sistema en situaciones reales, describiendo paso a paso el funcionamiento completo del software.

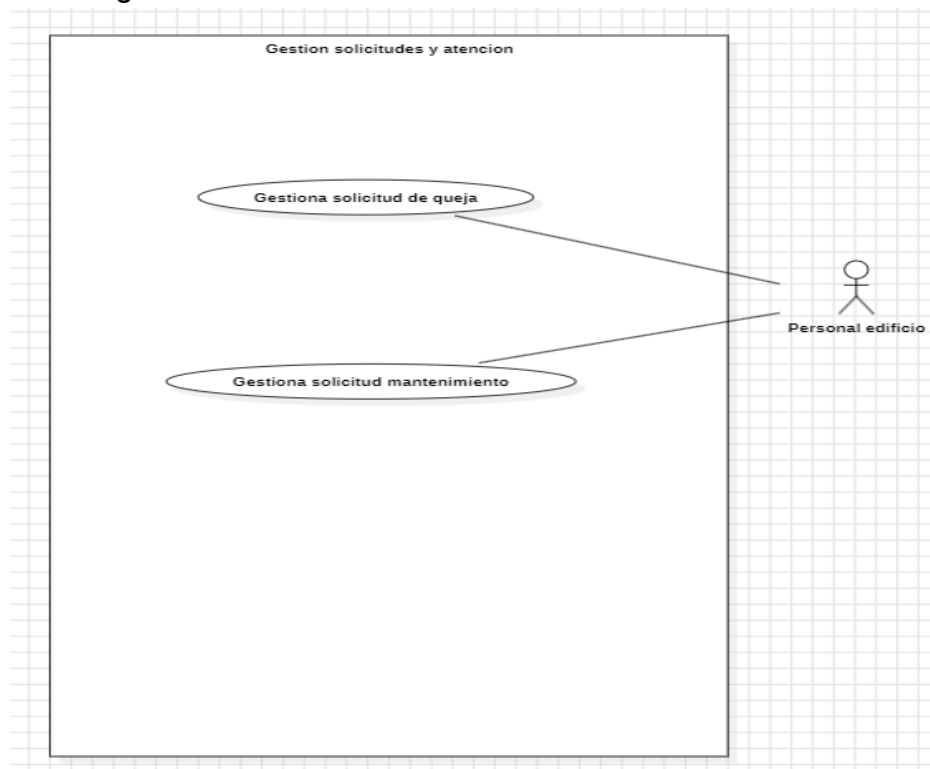
4.1.2. Actores

1. Administrador
2. Residentes (Propietario/Arrendatario)
3. Personal del Edificio
4. Comité administrador (Actor Secundario)
5. Sistema de pago (Webpay) (Actor externo)
6. Servicio notificaciones(Api) (Actor externo)

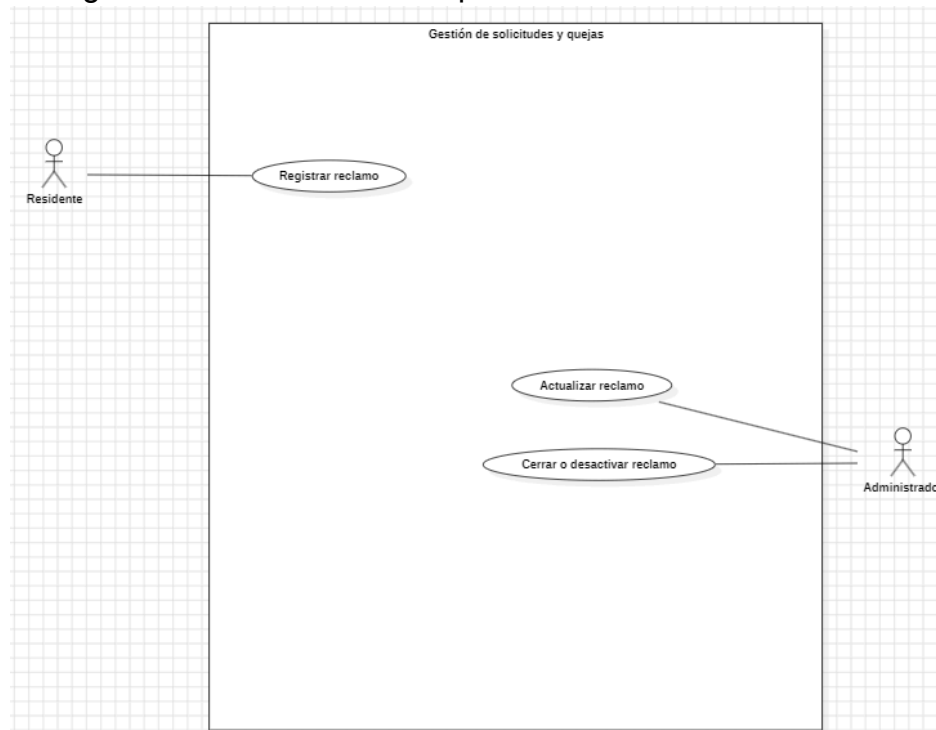
4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama caso de uso medio



4.1.5. Diagrama de casos de uso específicos



4.1.6. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Iniciar sesión	Todos
CU-001-002	Cerrar sesión	Todos
CU-001-003	Recuperar contraseña	Todos
CU-002-001	Registrar usuario	Residente /Administrador
CU-002-002	Actualizar usuario	Administrador
CU-002-003	Desactiva usuario	Administrador
CU-003-001	Configurar roles	Administrador
CU-003-002	Asignar permisos	Administrador
CU-003-003	Registrar residente	Administrador
CU-003-004	Actualizar residente	Administrador
CU-003-005	Desactivar/borrar residente	Administrador
CU-003-006	Registrar Unidad Habitacional	Administrador
CU-003-007	Configurar parámetros de cuotas	Administrador
CU-005-001	Generar cuotas automáticas	Sistema
CU-004-002	Registrar pago	Administrador
CU-004-003	Actualizar pago	Administrador

CU-004-004	Anular pago	Administrador
CU-006-001	Realizar pago webpay	Residente
CU-004-005	Registrar Gasto extra	Administrador
CU-004-006	Actualizar Gasto extra	Administrador
CU-004-007	Desactivar Gasto extra	Administrador
CU-005-002	Generar comprobante de Pago	Sistema
CU-005-003	Actualizar estado de morosidad	Sistema
CU-004-008	Registrar configuración de notificaciones	Administrador
CU-004-009	Actualizar configuración de notificaciones	Administrador
CU-004-010	Activar notificaciones	Administrador
CU-004-011	Desactivar notificaciones	Administrador
CU-004-012	Programar notificaciones	Administrador
CU-005-004	Enviar notificaciones	Sistema
CU-004-013	Consultar historial de comunicaciones	Administrador
CU-006-002	Registrar solicitud de mantenimiento	Residente
CU-007-001	Actualizar solicitud de mantenimiento	Personal
CU-007-002	Cerrar solicitud de mantenimiento	Personal
CU-004-014	Registrar reclamo	Administrador
CU-004-015	Actualizar reclamo	Administrador
CU-004-016	Cerrar reclamo	Administrador
CU-004-017	Generar informe general	Administrador
CU-004-018	General reporte financiero	Administrador
CU-004-019	Programar reportes automáticos	Administrador/ Sistema

4.1.7. Especificación de casos de uso

Caso de Uso	Registrar reclamo	Identificador: CU-004-014
Actores	Residente / Administrador/ Sistema	
Tipo	Primario	
Referencias	El sistema debe permitir el registro, Actualización y cierre (desactivación) de reclamos.	
Precondición	Creación y habilitación/activación en la cuenta del usuario al momento de ingresar reclamo	
Postcondición	La solicitud se registra en el sistema y pasa a tener un ESTADO el cual se actualiza dependiendo del cumplimiento al reclamo o desactivación según estime el personal a cargo de la solicitud	
Descripción	Una funcionalidad para residentes el cual les permite registrar reclamos y ver el estado en el que se encuentran dependiendo de la respuesta del personal el cual conlleva a una eventual solución a su reclamo o desactivación	
Resumen	Funcionalidad para usuarios residentes el cual les permite comunicar una queja o reclamo para la organización con fin de mejorar el área en cual es deficiente	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
5	Personal/Sistema	Toma la solicitud genera un estado de inicio para el reclamo actualiza el estado según tome decisión de cara a la solución
[Personal toma el reclamo Sistema genera un estado inicial(Enviado) Personal trabaja en el reclamo Personal actualiza el estado Sistema actualiza el estado (en Proceso, Desactivación/Cierre)]		

CURSO ALTERNATIVO

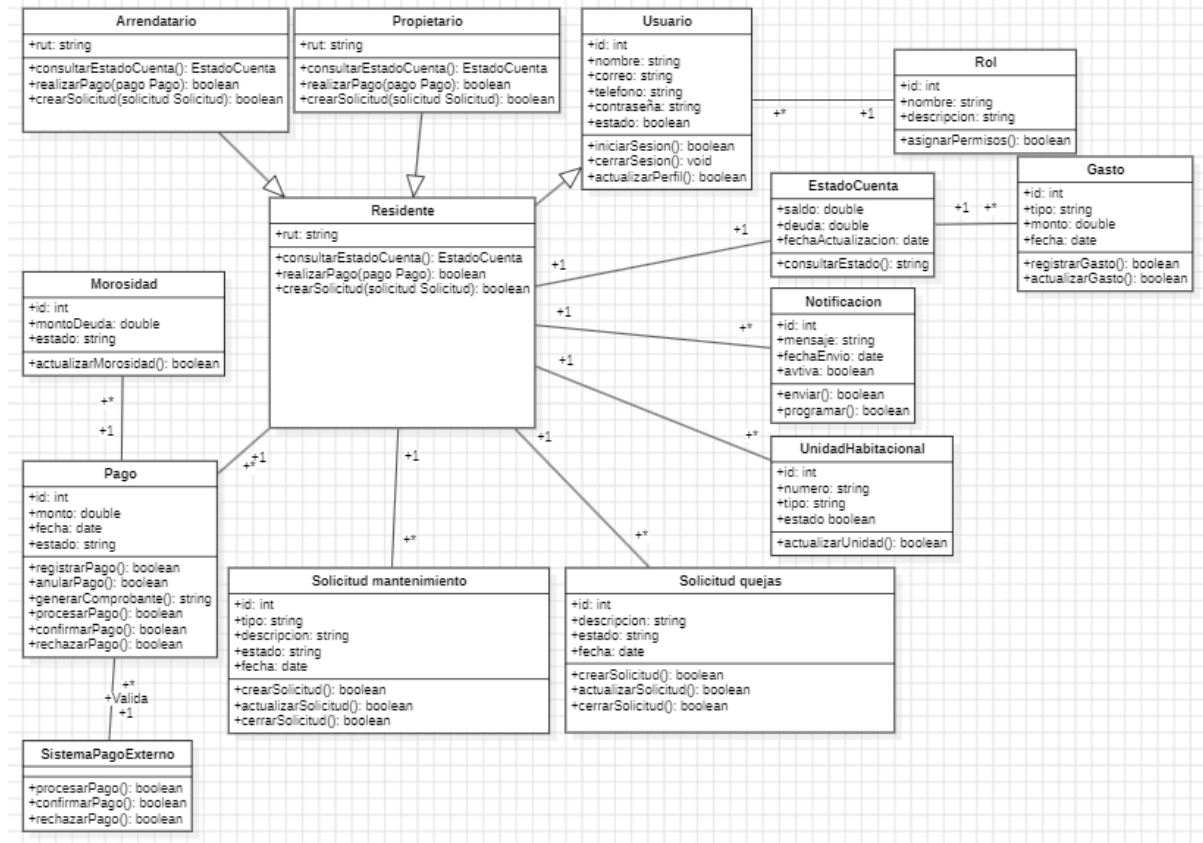
Nro.	Descripción de acciones alternas
6	Acciones secundarias de los actores en caso de no llevar el reclamo en su curso normal ej:Errores ortográficos (no entendimiento del reclamo), Reclamo fuera de lugar o sin sentido(El reclamo no tiene una base clara y no tiene intención de informar una problemática)
[Personal toma el reclamo Sistema genera un estado inicial(Enviado) Personal nota inconsistencia y sin lógica el reclamo Personal Cierra el reclamo Personal actualiza y desactiva(Estado) Sistema actualiza el estado]	

4.2. VISTA LÓGICA

4.2.1. Propósito:

Muestra la estructura interna del sistema, representando las clases, sus relaciones y cómo se organiza la lógica del software.

4.2.2. Diagrama de clases:



4.2.3. Descripción diagrama de clases:

El sistema se estructura en torno a la clase base 'Usuario', de la que heredan 'Administrador' y 'Residente'.

Las clases 'Pago', 'EstadoCuenta', 'Morosidad' y 'Gasto' conforman lo que sería el núcleo financiero, registrando transacciones, saldos, deudas.

'Solicitud' gestiona el ciclo de vida de reclamos y mantenciones.

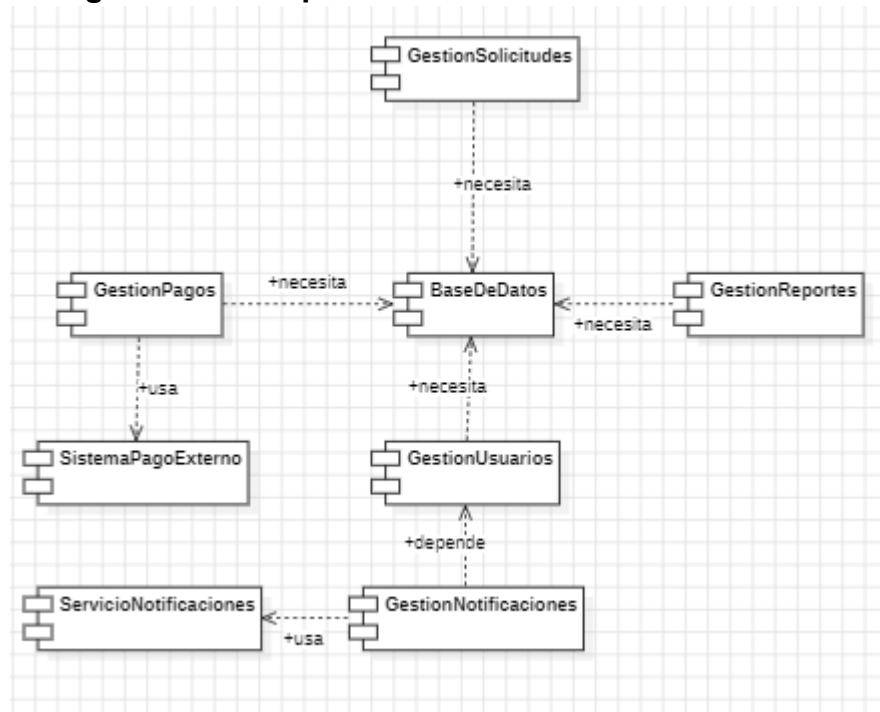
'SistemaPagoExterno' encapsula integración Webpay y 'Reporte' reúne la información financiera para su exportación.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito:

Muestra cómo se organiza técnicamente el software en módulos paquetes y componentes durante su desarrollo e implementación

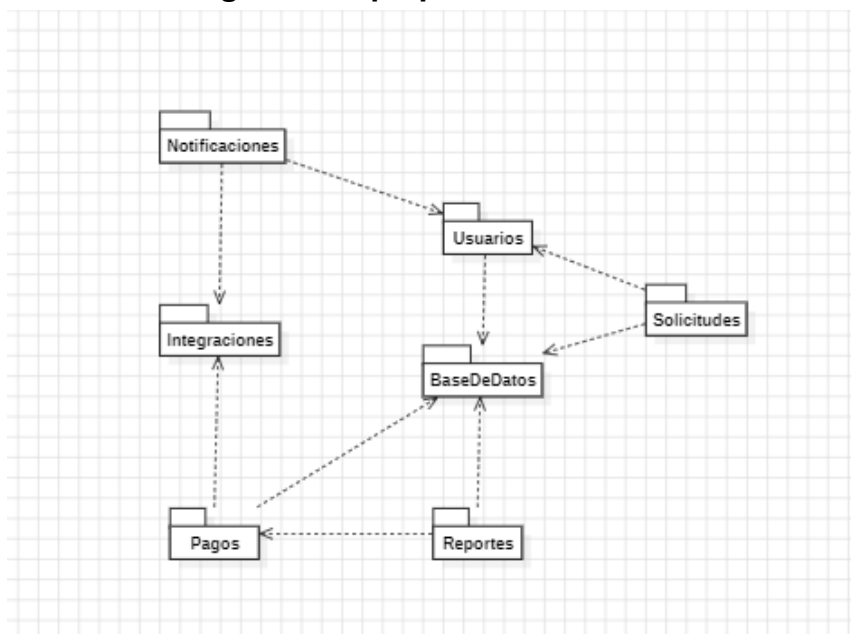
4.3.2. Diagrama de componente:



4.3.3. Descripción diagrama de componente:

Representa la estructura modular del sistema, mostrando los principales componentes funcionales y sus dependencias con la base de datos y servicios externos, como el sistema de pagos y el servicio de notificaciones.

4.3.4. Diagrama de paquete:



4.3.5. Descripción diagrama de paquete:

Representa la organización lógica del sistema, agrupando sus funcionalidades en módulos relacionados y mostrando las dependencias existentes entre ellos y la base de datos.

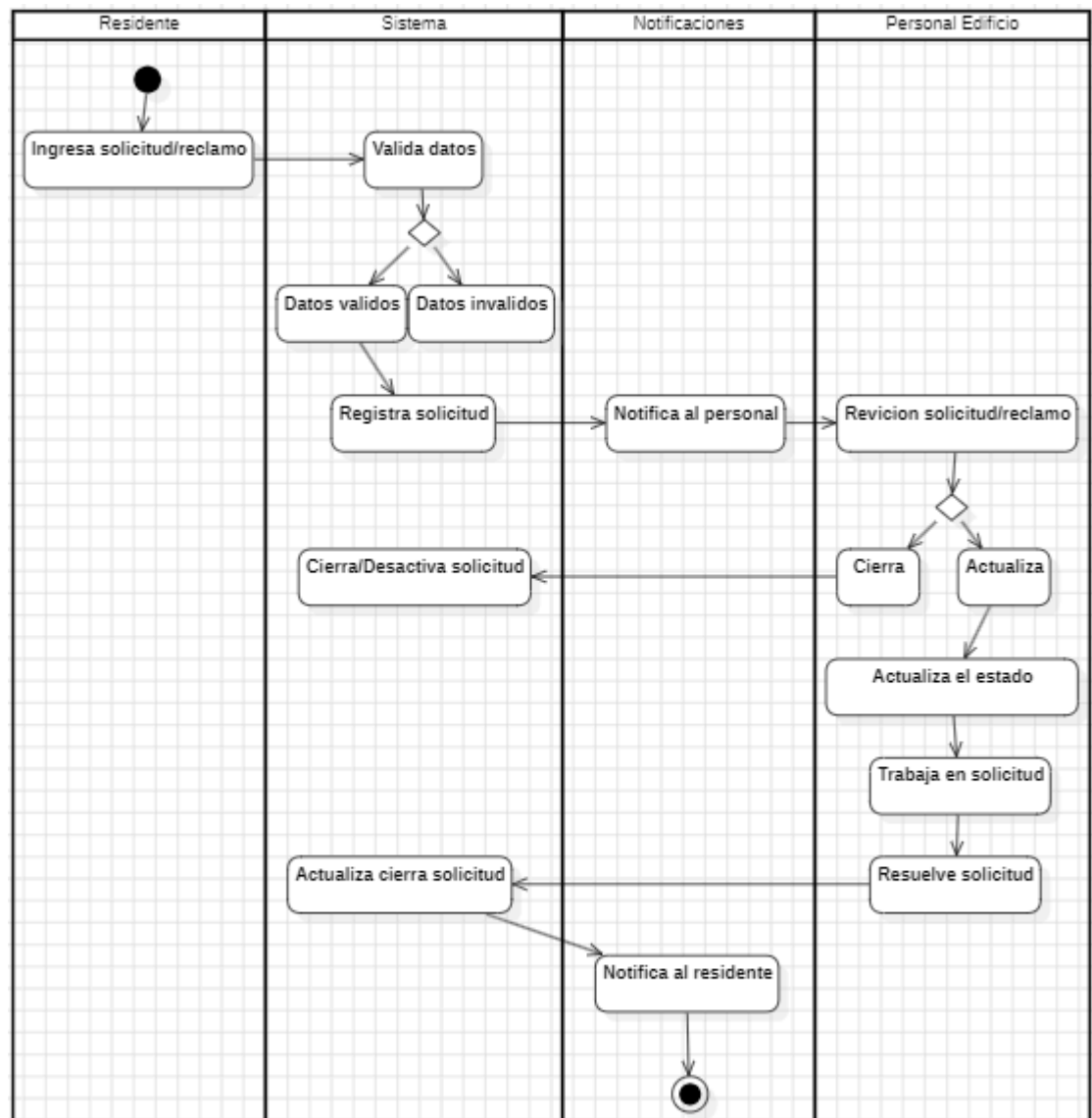
El paquete 'BaseDeDatos' es el núcleo del que dependen los demás, el paquete 'Integraciones' conecta con sistemas externos como webpay.

4.4. VISTA DE PROCESOS

4.4.1. Propósito:

Explicar cómo funciona el sistema paso a paso a través de sus procesos y flujos de trabajo.

4.4.2. Diagrama de actividad:



4.4.3. Descripción diagrama de actividad:

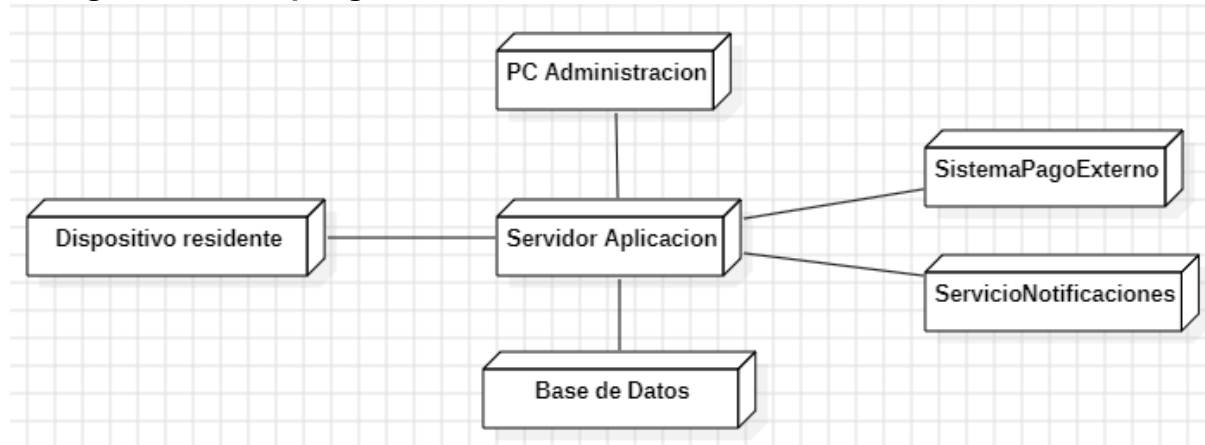
Representa el flujo de gestión de solicitudes y reclamos, mostrando las actividades, validaciones y decisiones involucradas en el registro, atención, actualización y cierre de solicitudes dentro del sistema.

4.5. VISTA FÍSICA

4.5.1. Propósito:

Despliega e instala el sistema en la infraestructura física, representando servidores, dispositivos, bases de datos y conexiones de red.

4.5.2. Diagrama de despliegue:



4.5.3. Descripción diagrama de despliegue:

Representa la infraestructura física del sistema, mostrando los dispositivos y servidores involucrados en la ejecución de la aplicación, junto con la comunicación con la base de datos y servicios externos de pagos y notificaciones.

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt